

Практическая работа 1.3

Авторизация в Web-приложениях Node.js

Цель работы

Данная практическая работа направлена на отработку и закрепление навыков создания примитивных web-приложений с использованием фреймворка express.js и шаблонизатора на выбор, а также изучение вариантов реализации авторизации пользователей в express.js приложениях.

Задание

В рамках выполнения практической работы требуется выполнить следующие работы и продемонстрировать полученные навыки: – На основе приложения разработанного в практической работе 1.2 реализовать аналогичное приложение, поддерживающее авторизацию пользователей – Реализовать логику обработки на сервере согласно варианту задания

Содержание отчета

- Цель работы
- Условие задачи, которая будет решаться
- Скриншоты получившегося приложения
- Исходный код решения

Распределение вариантов задач

Первая буква фамилии	Задачи
А,Б,В,Г,Д,	1
Е,Ж,З,И,К,Л,	2
М,Н,О,П,Р,	3
С,Т, У,Ф,Х,Ц,	4
Ч,Ш,Щ,Ю,Я	5

Оценивание

Практическая работа оценивается в общей сложности в 8 баллов. При этом:

- Задача - 7 баллов
- Отчет - 1 балл
- Бонус - 2 дополнительных балла

Задача представляются преподавателю на занятиях. По коду задачи могут быть заданы вопросы.

Оценивается работоспособность решения, а также общее понимание кода студентом. В отчете оценивается наличие всех указанных разделов.

Задачи

Задача 1: Калькулятор калорийности рациона

Необходимо реализовать веб-калькулятор для подсчета калорийности блюда. На основной странице должны находиться поля: название продукта (выпадающий список или текстовое поле с автодополнением, данные берутся из файла), поле для ввода веса продукта в граммах. Также должна быть кнопка "Рассчитать". При нажатии сервер вычисляет общую калорийность на основе данных из файла и возвращает результат пользователю. На отдельной странице должен выводиться справочник продуктов с их калорийностью (на 100г).

- **Хранение данных:** Справочник продуктов хранится в JSON файле на сервере.
- Приложение должно поддерживать авторизацию пользователей. Список пользователей с паролями храниться в файле на сервере.

- Для каждого пользователя должен храниться и редактироваться его список продуктов (т.е. добавлять продукты могут все пользователи, но только себе).

Задача 2: Трекер багов (Bug Tracker)

Необходимо реализовать упрощенную систему отслеживания ошибок. На основной странице отображается список активных багов (описание, приоритет, дата создания). Присутствует форма для создания нового бага. Рядом с каждым багом в списке должна быть кнопка "Исправить". После нажатия баг помечается как закрытый и исчезает из основного списка. На отдельной странице должен отображаться архив исправленных багов. Ссылка на архив доступна с главной страницы. Данные сохраняются в файл на диске.

- **Хранение данных:** Массив объектов в JSON файле.
- Приложение должно поддерживать авторизацию пользователей. Список пользователей с паролями храниться в файле на сервере.
- Каждый пользователь видит и редактирует только свои баги.

Задача 3: Система сбора обратной связи (Feedback)

Необходимо реализовать форму для сбора отзывов о курсе. Вопросы для отзыва (например, "Что понравилось?", "Что улучшить?", "Оценка от 1 до 5") должны храниться в конфигурационном файле на сервере. При открытии приложения пользователь видит форму с этими вопросами. Ответы сохраняются в файл логов. На отдельной странице реализовать дашборд для преподавателя: средняя оценка и количество полученных отзывов.

- **Хранение данных:** Вопросы в одном файле, ответы в другом.
- Приложение должно поддерживать авторизацию пользователей. Список пользователей с паролями храниться в файле на сервере. У пользователей есть роль студент или преподаватель.
- На страницы дашборда преподавателя могут заходить только преподаватели, а в опрос только студенты.

Задача 4: Бронирование переговорных комнат

Необходимо реализовать приложение для бронирования рабочих помещений. На основной странице форма: выбор комнаты (из списка), дата и время начала встречи, длительность (в часах). Запись сохраняется в файл. Если на выбранное время комната уже занята (или время встречи пересекается с существующей записью), приложение

должно возвращать ошибку. На отдельной странице выводится расписание занятости комнат на ближайшие дни. Прошедшие события скрываются.

- **Хранение данных:** Журнал бронирований в JSON файле.
- Приложение должно поддерживать авторизацию пользователей. Список пользователей с паролями храниться в файле на сервере.
- Пользователь видит все бронирования. К бронированию добавляется имя пользователя, который его сделал. Пользователь может редактировать и удалять только свои бронирования.

Задача 5: Каталог домашних рецептов

Необходимо реализовать приложение "Книга рецептов". На основной странице выводится список имеющихся рецептов (название, краткое описание). На дополнительной странице находится форма для добавления нового рецепта (название, ингредиенты, инструкция). Рецепты сохраняются в файл на диске.

- **Хранение данных:** Рецепты в JSON файле.
- Приложение должно поддерживать авторизацию пользователей. Список пользователей с паролями храниться в файле на сервере.
- Каждый пользователь видит и редактирует только свои рецепты.